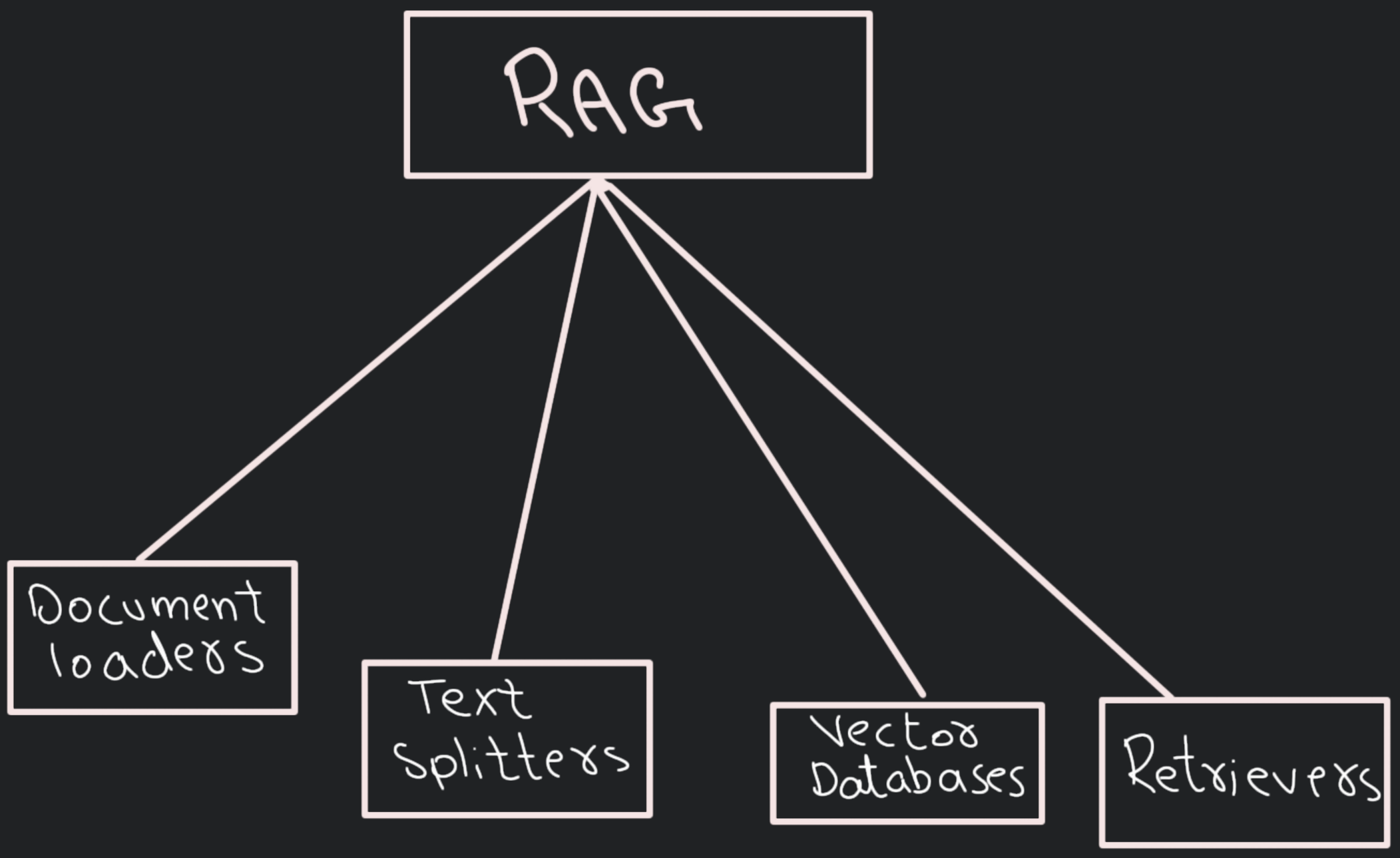
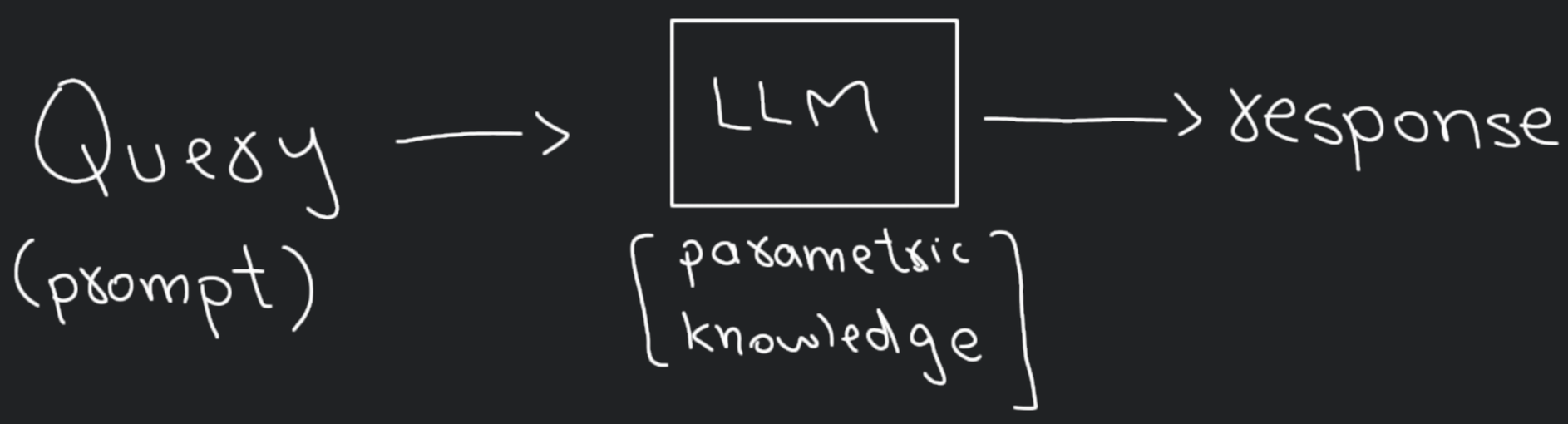


Why?



LLM:

- can't prompt on private data
- can't answer recent questions
- hallucination (factually incorrect)

Solution is:

Fine-Tuning

Train pre-trained LLM on smaller domain specific dataset

- Supervised fine-tuning
  - train on labeled dataset

- Continued pre training
  - unsupervised

- RLHF

Steps:

1) Collect data

2) Choose a Method (FT, LoRA/QLoRA etc)

3) Train for a few epochs

4) Evaluate and safety test

Problems:

1) Training LLM is computationaly expensive

2) Requires strong technical expertise

3) Requires training again and again whenever information changes to keep model up-to-date

In-context Learning (Few shot prompting)

Core capability of LLMs like GPT-3/4, Claude, and Llama, where model learns to solve a task purely by seeing examples in the prompt- without updating its weight

. Emergent property of a LLM
  ↳ behavior or ability that suddenly appears in a system when it reaches a certain scale or complexity - even though it was not explicitly programmed or

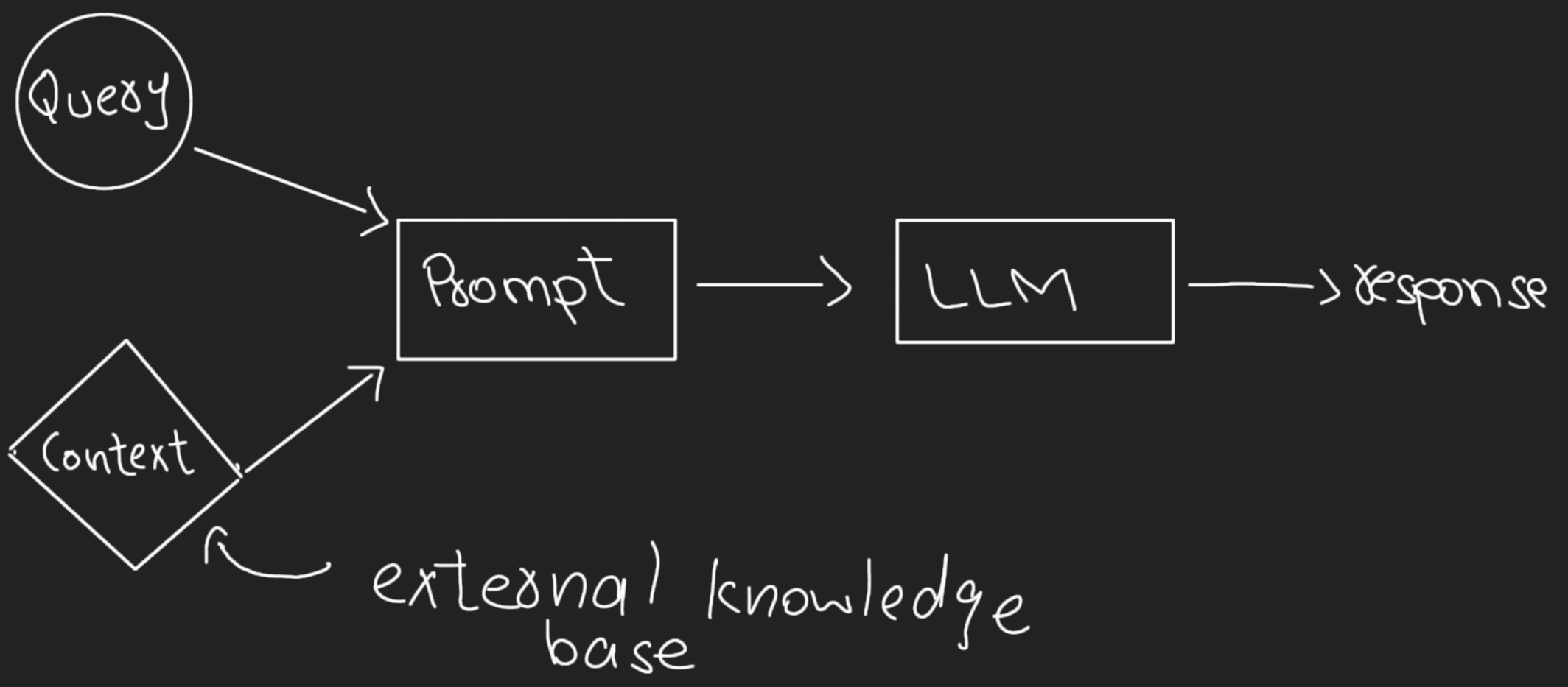
expected from the individual components

→ Instead of just example tasks, retrieve background information, facts, documents, product manuals, etc

→ Inject that into the prompt to augment the model's knowledge

RAG

A way to make a LLM smarter by giving it extra information at the time you ask your question



Understanding RAG

RAG → Information Retrieval + Text Generation

Steps:

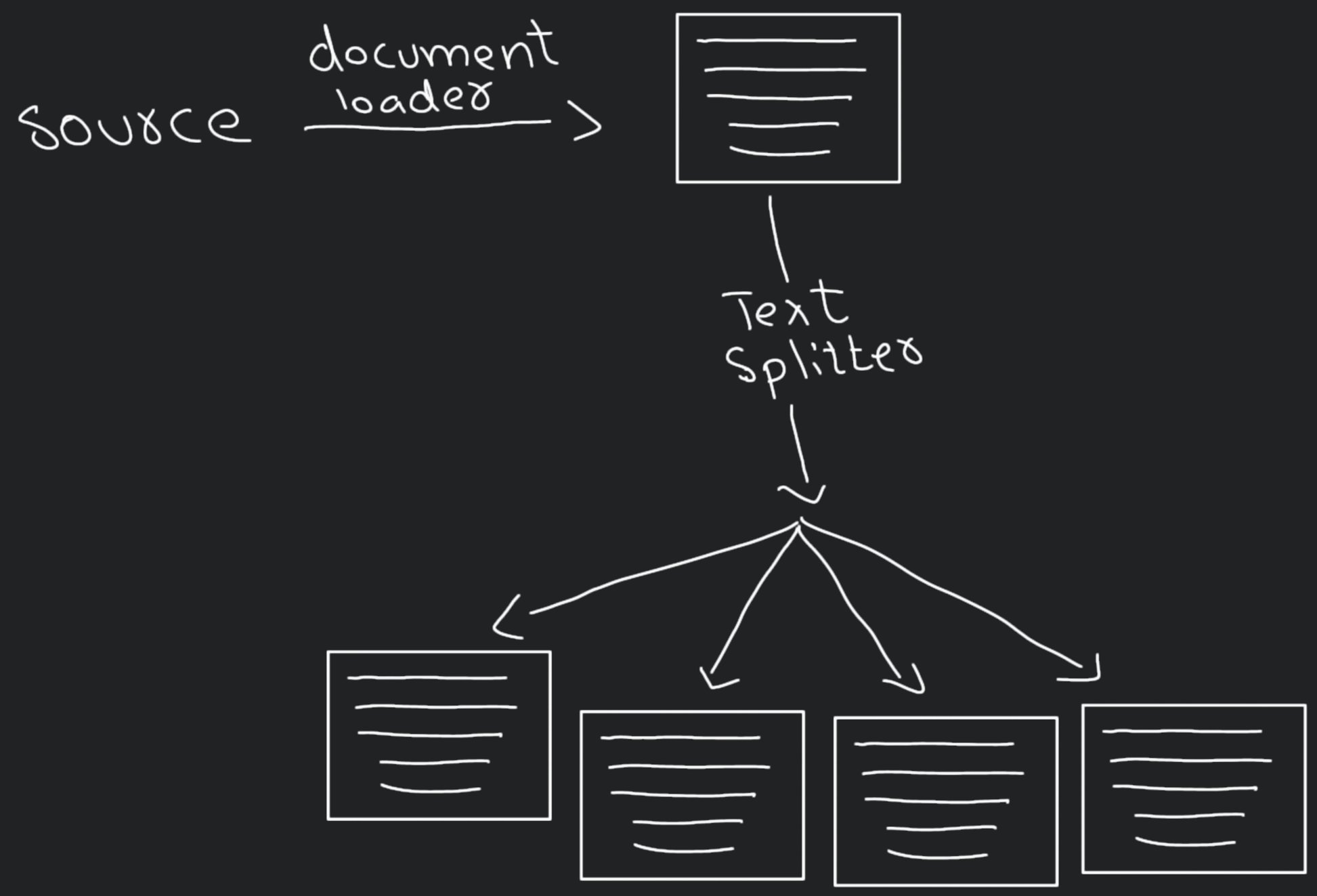
- Indexing (External knowledge Base)
- Retrival
- Augmentation (Prompt -> Query + context)
- Generation

Indexing

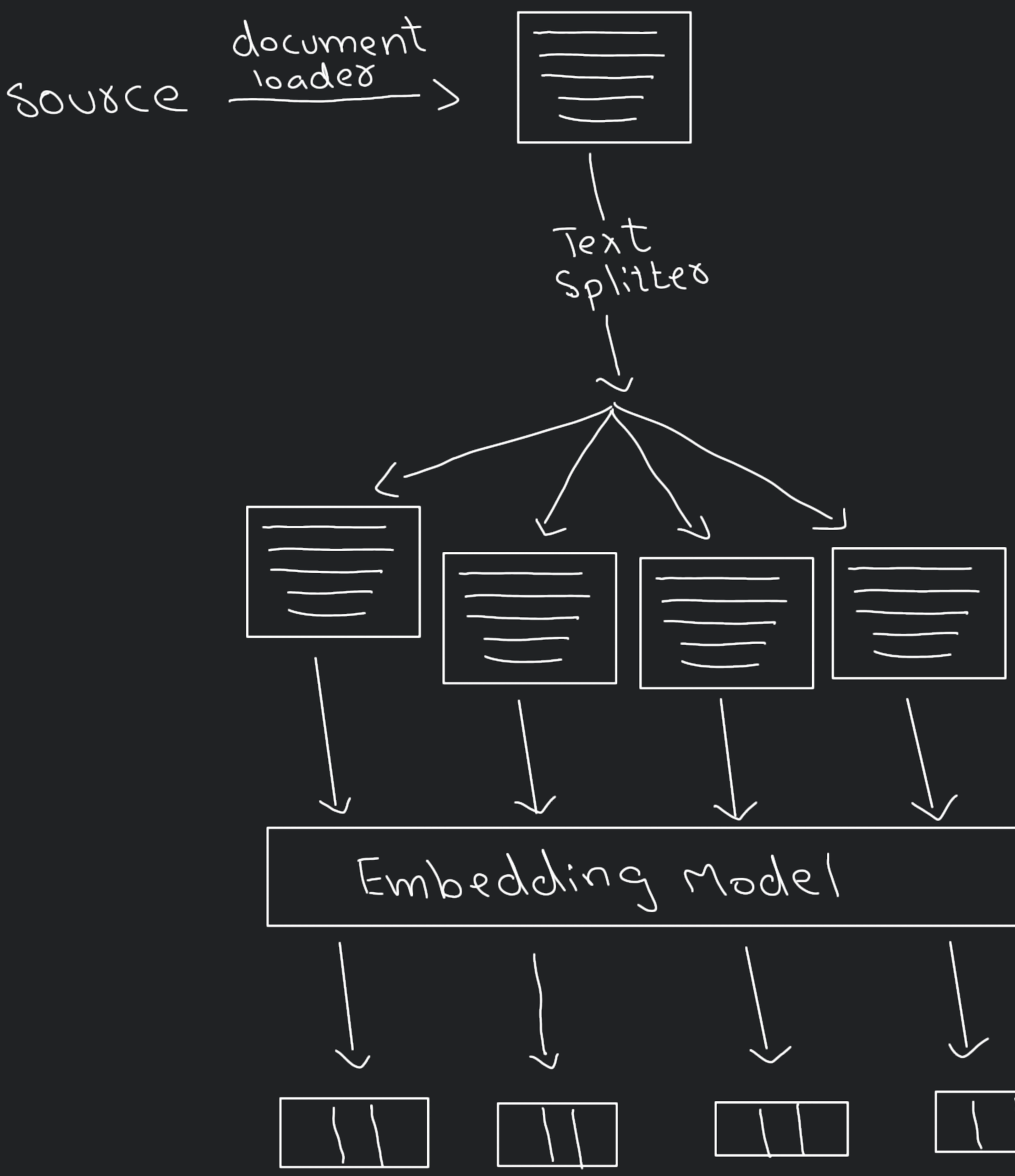
process of preparing your knowledge base so that it can be efficiently searched at query time

1. Document Ingestion - load source knowledge into memory (using document loaders)

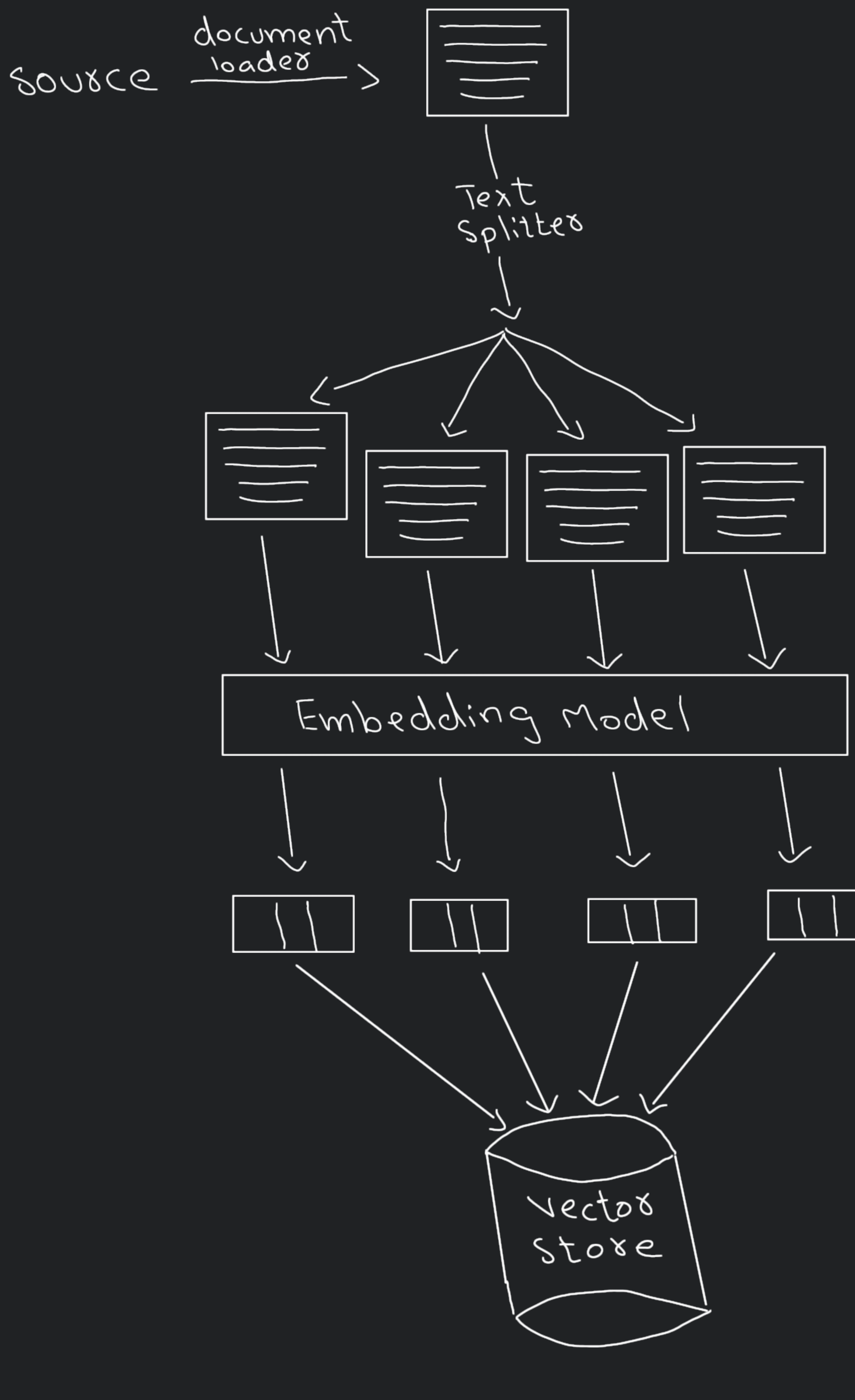
2. Text Chunking - break large docs into small, semantically meaningful chunks
(Text splitting)



3. Embedding Generation - convert each chunk into a dense vector (embedding) that captures its meaning



4. Storage in a Vector Store — Store the vectors along with original chunk text + metadata in a vector database


Source
document loader
Text Splitter
Embedding Model
Vector Store

Retrieval

Real-time process of finding the most relevant pieces of information from pre-built index (created during indexing) based on user's question

1) Convert query into embedding vector

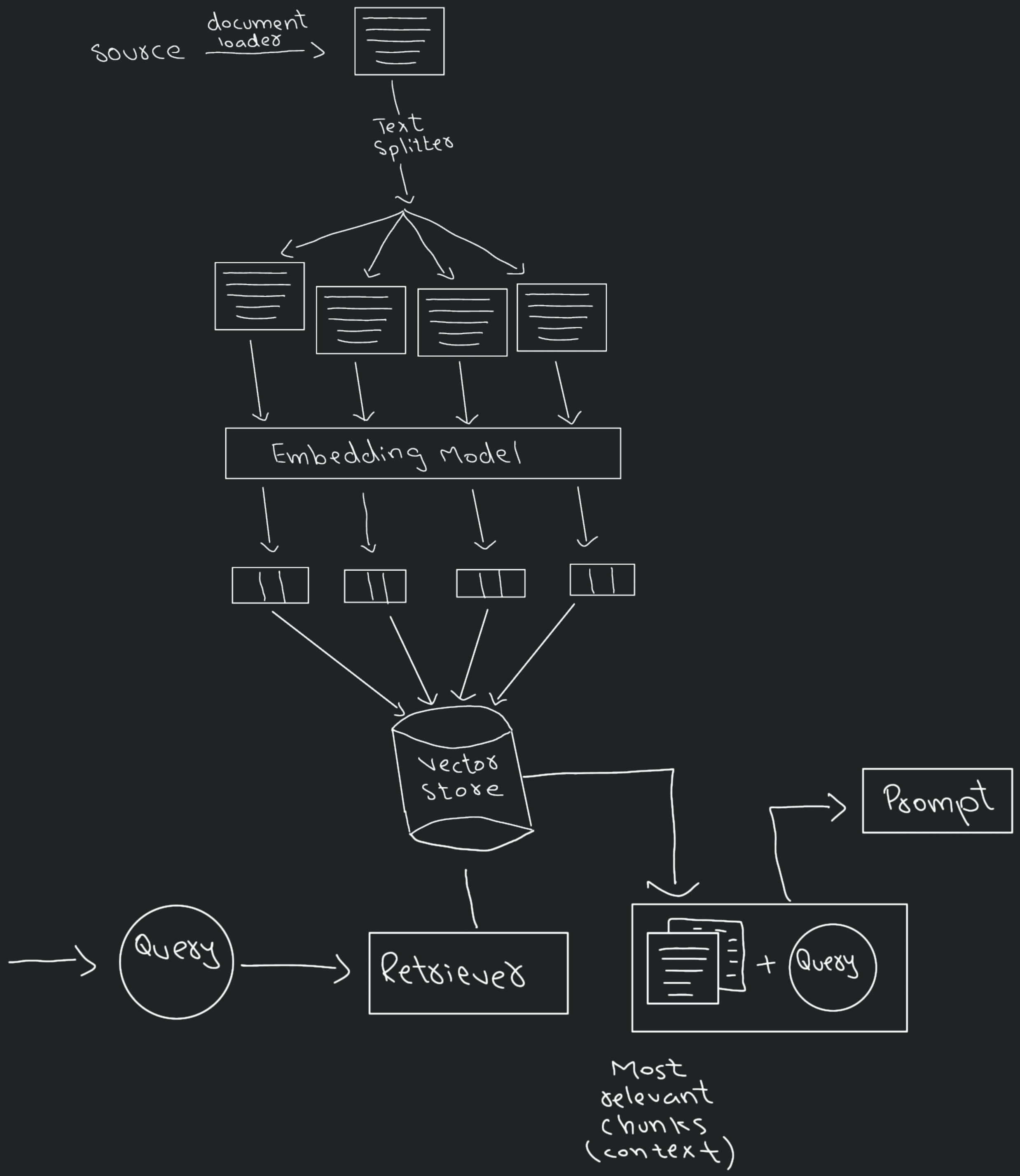
2) Semantic Search

3) Ranking (order chunks)

4) Fetch top results (chunks

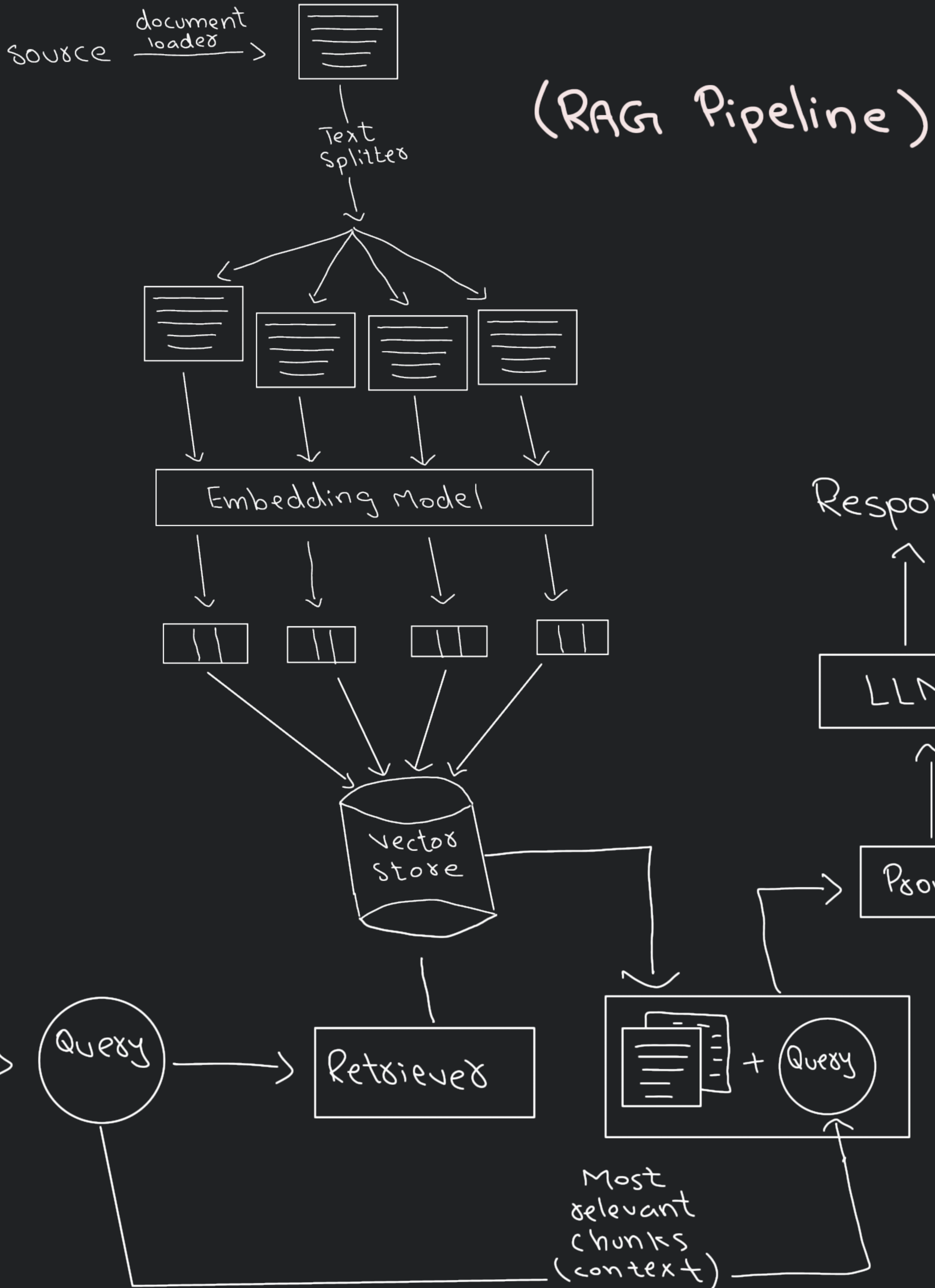
Augmentation

Create prompt from query and context



Generation

Final Step where a LLM uses the user's query and the retrieved and augmented context to generate a response



How it solves those 3 problems:

1) Private data

—> You provide your own data in knowledge base

2) Recent Information

—> just need to update knowledge base

3) Hallucinations

—> we are providing the exact context and asking the LLM to question ONLY from provided context. If the context is insufficient tell the LLM to say "you don't know"

Improvements

1) UI based enhancements

2) Evaluation
   a. Ragas
   b. LangSmith

3) Indexing
   a. Document Ingestion --> fix errors / preprocessing
   b. Text Splitting
   c. Vector store

4) Retrieval
   a. Pre-Retrieval
      i. Query rewriting using LLM
      ii. Multi-query generation
      iii. Domain aware routing

   b. During Retrieval
      i. MMR
      ii. Hybrid Retrieval
      iii. Reranking

c. Post Retrieval
    i. Contextual Compression

5) Augmentation
    a. Prompt Templating
    b. Answer grounding
    c. Context window optimization

6) Generation
    a. Answer with Citation
    b. Guard railing

7) System Design
    a. Multimodal RAG
    b. Agentic RAG
    c. Memory based RAG